Business Intelligence Systems

INFO 4555 - Fall 2025

Web Services



What is an API?

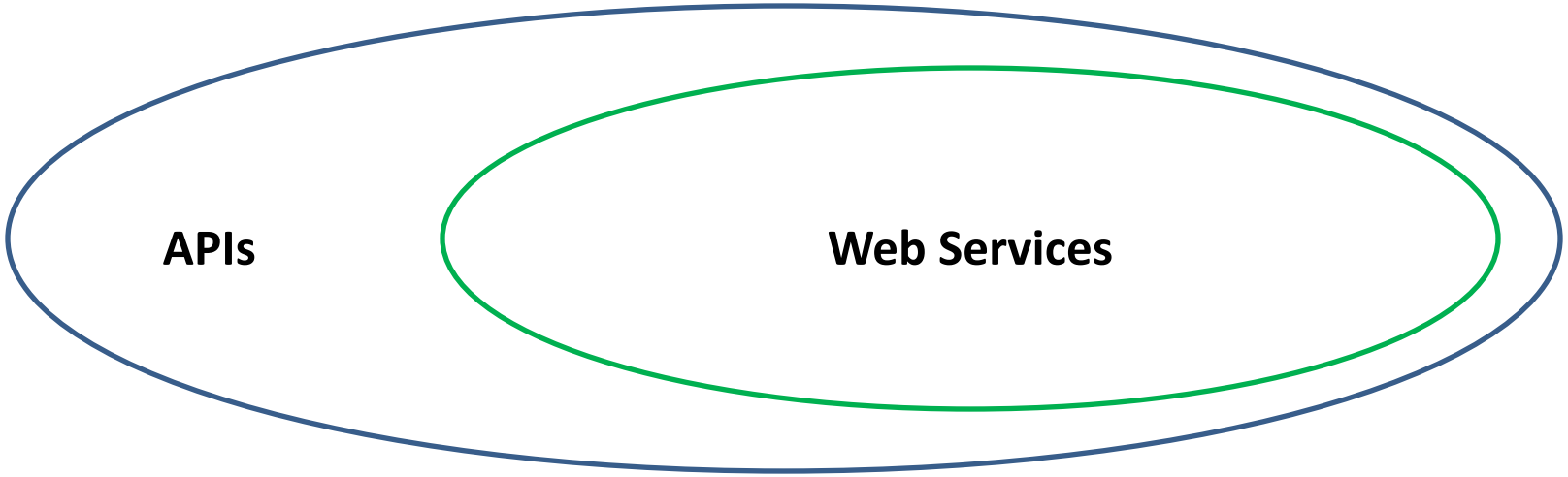
- Exposed rules and protocols for an application's functionality
- Enables an application to share its data or functionality
- Access to these services might be via programmatic execution e.g. via the use of Java Archive (JAR) collections, C/C++ interface specs (.h files), Python library APIs, or other specification documents

- Most modern API accesses are via the Internet
- API traffic accounts for ~71% of all web traffic

Source: https://www.imperva.com/resources/resource-library/reports/the-state-of-api-security-in-2024/

What is a Web Service?

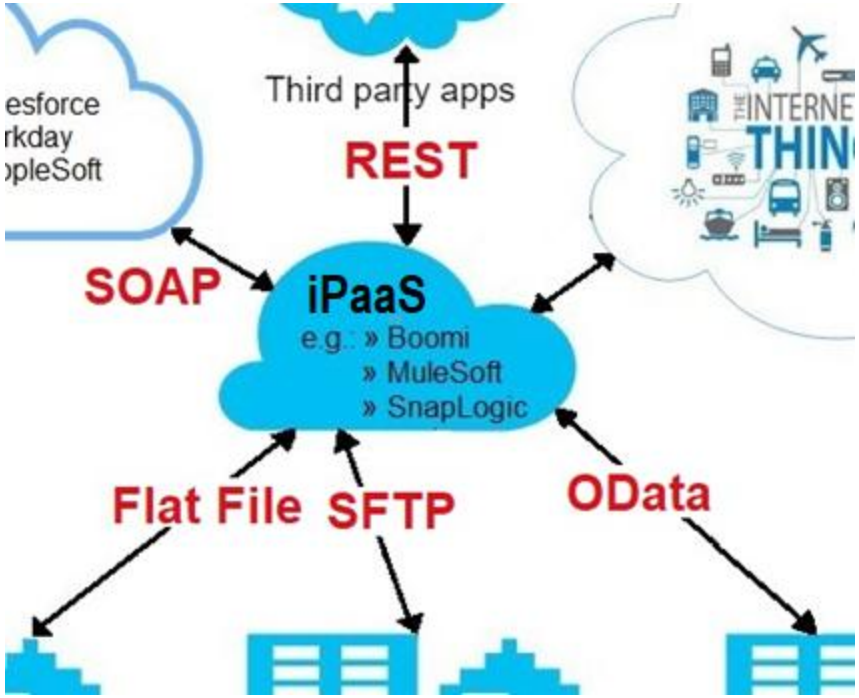
- Web services refer specifically to APIs accessed over a network, especially the Internet



Web Service Data Exchange Methods

While significant data exchange remain via Flat File and Secure File Transfer Protocol (*SFTP)*, there are currently several standard API protocols over which most Web Service conversations occur:

- REST (by far most common)
- SOAP
- GraphQL
- WebSockets



the vast majority of "-as a Service" solutions expose and depend on web services

Web Service Interface Signatures

Different Web Service Protocols use interchange formats specific to the way their requests and responses are to be “packaged”

REST exchanges are very flexible in what they can allow for requests and responses, but typically JSON or XML

OData leverages the flexibility of REST but reduces randomness by formalizing their exposed request/response rules (often resembling SQL operations)

SOAP expects inputs and outputs to be formatted as XML

GraphQL has its own syntax, but JSON is used when serving over HTTP

WebSockets are transmitted over TCP for real-time 2-way communication. Message syntax is flexible



REST (REpresentational State Transfer) Protocol

REST refers to the protocol architecture

A RESTful API is one utilizing this protocol

RESTful requests…

- Use standard HTTP list of verbs: **GET**, **HEAD**, **POST**, **PUT** and **DELETE**
- Have light-weight request/response interfaces
- Are stateless (not order dependent)
- Require agreed upon methods with agreed upon behaviors

REST Requests

Consist of

- HTTP method (verb)
- URL (endpoint)
- Path parameters
- Query parameters
- Headers
- Request body (for POST or PUT)

Methods (verbs)

- **GET**: Retrieve data from the server.
- **HEAD:** Return only the header data without a message body
- **POST**: Submit data to be processed (e.g., create a new resource).
- **PUT**: Update or replace an existing resource on the server
- **DELETE**: Remove a resource.

URL (endpoint)

The URL (Uniform Resource Locator) represents the address of the resource you are interacting with. It's usually structured as:

- Base URL: The domain of the server hosting the API.
- Path: Specifies the resource (e.g., /users, /posts).

  Example: https://api.example.com/users

- Example from homework assignment:

https://cornell-icc-openai.openai.azure.com/openai/deployments/dall-e-3/images/generations

Path Parameters

Path parameters are part of the URL path and usually specify the specific resource you are interacting with.

Example: https://api.example.com/users/{user_id}

{user_id} is the path parameter and would be replaced with an actual user ID, e.g. https://api.example.com/users/123

Query Parameters

Query parameters are appended to the URL after a ? and are typically used to filter or refine results.

Example: https://api.example.com/users?limit=10&sort=desc

limit=10: Limits the response to 10 results.
sort=desc: Sorts the results in descending order.

Query parameters are key-value pairs, and multiple parameters are separated by &.

HTTP Headers

HTTP headers provide metadata about the request or response, such as content type, authorization, and other parameters.

Common Headers:

- Content-Type: Specifies the format of the data being sent (e.g. application/json, application/xml)
- Authorization: Includes credentials for accessing protected resources (e.g., Bearer Token (OAuth), API Key, Basic Authentication (username & password)).
- Accept: Tells the server what type of response the client expects (e.g., application/json or application/xml)

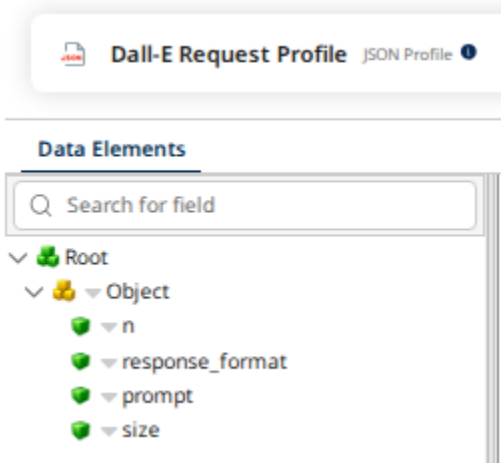
https://en.wikipedia.org/wiki/List_of_HTTP_header_fields

Request Body

The body contains the data that the client sends with POST or PUT requests, in JSON or XML format

Example from homework (POST request in JSON format)

```
{
"prompt": "A cow playing an alpine horn in the Swiss Alp mountains",
"size": "1024x1024",
"n": "1",
"response format":"b64_json"
}
```



prompt: the prompt being generated

size: the size of the generated image

n: the number of images to be generated by DALL-E 3

response_format: the formatting of the generated image

Response body

Response body: the server response to the given query.

Returned data is specified by the endpoint queried; different endpoints return different pre-defined sets of data

Example from homework (JSON format):

```
{
   "created": 1727063997,
   "data": [
      {
         "b64_json": {image data}
      }]
}
```

HTTP Status Codes

HTTP status codes indicate the result of the API request.

- **Common Status Codes**:
  - **200 OK**: The request was successful
  - **400 Bad Request**: There was an error in the request
  - **401 Unauthorized**: Authentication failed or was missing
  - **403 Forbidden**: Insufficient permissions to access the given resource
  - **404 Not Found**: The requested resource doesn't exist
  - **500 Internal Server Error**: A server error occurred

https://en.wikipedia.org/wiki/List_of_HTTP_status_codes

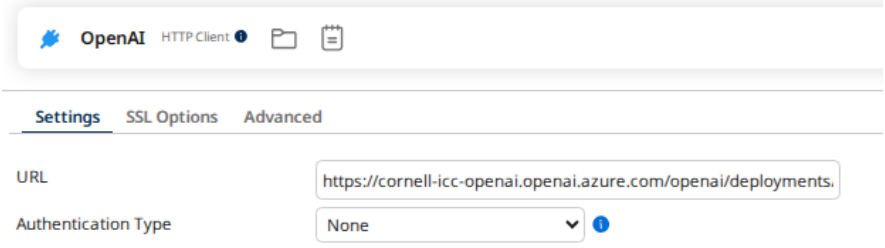
REST API in Boomi

- Access REST APIs via HTTP Client shape
- Build and expose APIs using API Management
  - SOAP/REST/OData Web Service APIs for triggered integrations
  - Role-based authentication
  - Allows a user without access to Boomi to run an integration using a browser, app, or code
  - Multiple endpoint and parameter definition
  - Support for API Gateway

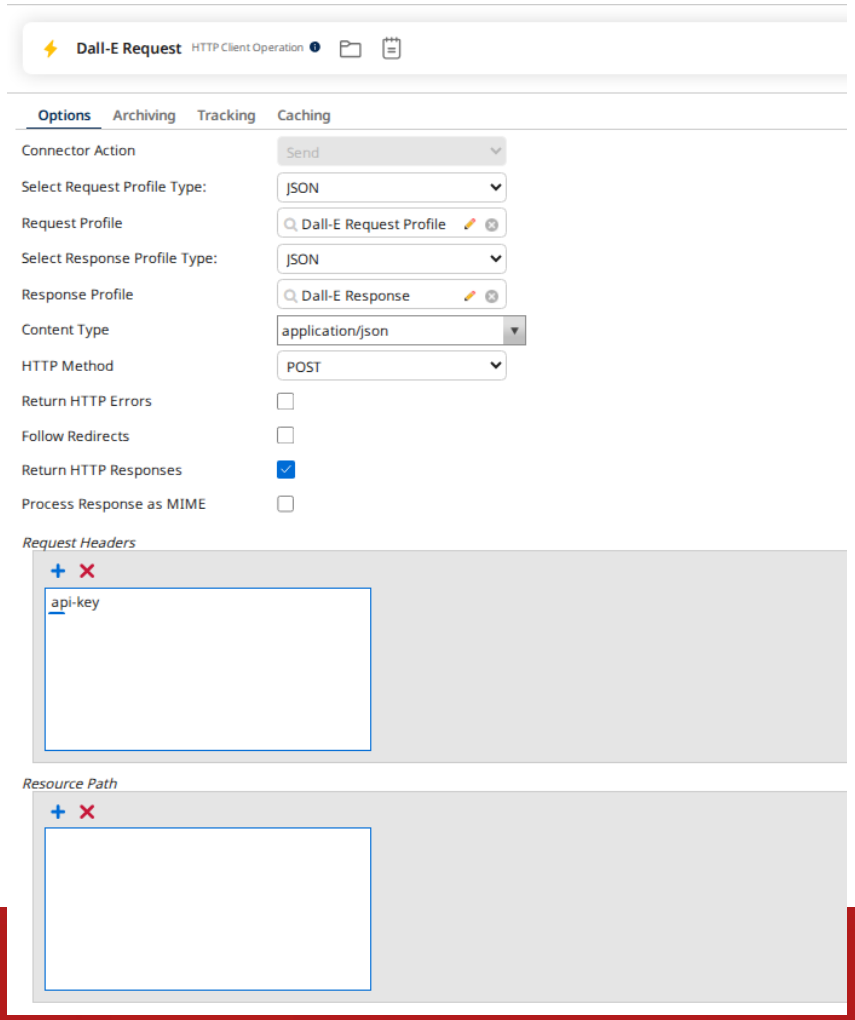
HTTP Client Shape

Highly flexible, easily configurable Shape with support for GET, POST, PUT and DELETE HTTP methods

Connection:

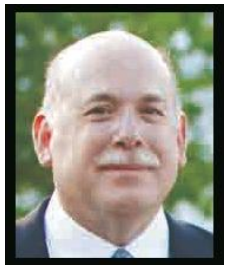


Operation:

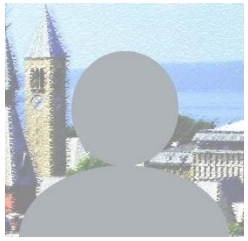


Example of API managed by Boomi: REST Service to retrieve person's image

Note: The parameter here is of type string and returned object is MIME type image/jpg



Example 1: Parameter value is "jap54":
https://api-test.integrations.cornell.edu/ws/rest/getPersonImage/netId/jap54

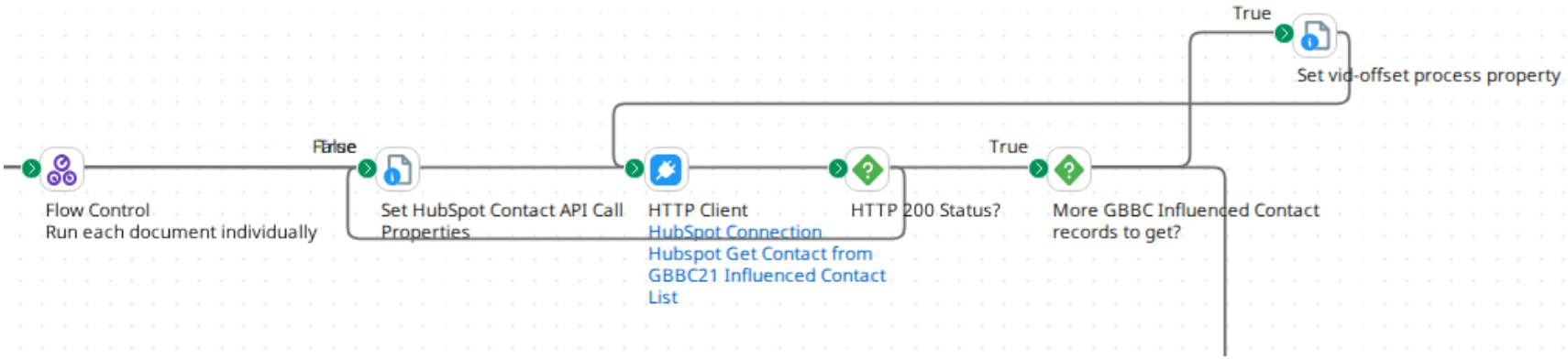


Example 2: Parameter value is "jap54X":
https://api-test.integrations.cornell.edu/ws/rest/getPersonImage/netId/jap54X

- Underlies "Common Good" function to retrieve person's image
- Lightweight service used by Salesforce, mobile apps, etc.
- Verifies images are not suppressed (CU Police officers, students/staff "at risk")
- Returns suppressed image for invalid netIds

Pagination

- Large data size increases potential for timeouts or data loss
- For sufficiently large datasets, data is chunked into pages
- Multiple API calls are made to the same endpoint via a looping structure
- Each page can be marked numerically or with some ID, which is then
- API Response may contain information about the total number of records returned



OData (Open Data Protocol)

- The OData Protocol is an application-level protocol for interacting with data via RESTful web services
- Typical usage of ODATA is for accessing DATA (vs functional results) from a data store, whether it be in a database, flat file system, etc…
- OData is essentially “SQL for the web” using standard protocols like HTTP, JSON, etc., but implemented in the simpler REST style

OData by Example (and imagination)

Imagine  **a Travel Management System** with an underlying database that holds the detail for all of its travelers.

Contemplate the SQL that might be needed to retrieve all or parts of the stored detail.

NOW, picture sets of RESTful - simple Web Service - OData requests that can "wrap" the underlying traveler database access functions…

For example, retrieving the information for ALL of its travelers:

https://services.odata.org/v4/(S(0ujx3qru3djokaxrka1jknic))/TripPinServiceRW/People

(from www.odata.org)

OData Results

ALL of the traveler's info via a RESTful API request

{"@odata.context":"http://services.odata.org/V4/(S(zirb3lxmgzynebnvq5uhdtgx))/TripPinServiceRW/$metadata#People","@odata.nextLink":"https://services.odata.org/V4/(S(zirb3lxmgzynebnvq5uhdtgx))/TripPinServiceRW/People?%24skiptoken=8","value":[{"@odata.id":"http://services.odata.org/V4/(S(zirb3lxmgzynebnvq5uhdtgx))/TripPinServiceRW/People('russellwhyte')","@odata.etag":"W/\"08D61F10043BBCA1\"","@odata.editLink":"http://services.odata.org/V4/(S(zirb3lxmgzynebnvq5uhdtgx))/TripPinServiceRW/People('russellwhyte')","UserName":"russellwhyte","FirstName":"Russell","LastName":"Whyte","Emails":["Russell@example.com","Russell@contoso.com"],"AddressInfo":[{"Address":"187 Suffolk Ln.","City":{"CountryRegion":"United States","Name":"Boise","Region":"ID"}}],"Gender":"Male","Concurrency":636730550970465441},{"@odata.id":"http://services.odata.org/V4/(S(zirb3lxmgzynebnvq5uhdtgx))/TripPinServiceRW/People('scottketchum')","@odata.etag":"W/\"08D61F10043BBCA1\"","@odata.editLink":"http://services.odata.org/V4/(S(zirb3lxmgzynebnvq5uhdtgx))/TripPinServiceRW/People('scottketchum')","UserName":"scottketchum","FirstName":"Scott","LastName":"Ketchum","Emails":["Scott@example.com"],"AddressInfo":[{"Address":"2817 Milton Dr.","City":{"CountryRegion":"United States","Name":"Albuquerque","Region":"NM"}}],"Gender":"Male","Concurrency":636730550970465441},{"@odata.id":"http://services.odata.org/V4/(S(zirb3lxmgzynebnvq5uhdtgx))/TripPinServiceRW/People('ronaldmundy')","@odata.etag":"W/\"08D61F10043BBCA1\"","@odata.editLink":"http://services.odata.org/V4/(S(zirb3lxmgzynebnvq5uhdtgx))/TripPinServiceRW/People('ronaldmundy')","UserName":"ronaldmundy","FirstName":"Ronald","LastName":"Mundy","Emails":["Ronald@example.com","Ronald@contoso.com"],"AddressInfo":[],"Gender":"Male","Concurrency":636730550970465441},{"@odata.id":"http://services.odata.org/V4/(S(zirb3lxmgzynebnvq5uhdtgx))/TripPinServiceRW/People('javieralfred')","@odata.etag":"W/\"08D61F10043BBCA1\"","@odata.editLink":"http://services.odata.org/V4/(S(zirb3lxmgzynebnvq5uhdtgx))/TripPinServiceRW/People('javieralfred')","UserName":"javieralfred","FirstName":"Javier","LastName":"Alfred","Emails":["Javier@example.com","Javier@contoso.com"],"AddressInfo":[{"Address":"89 Jefferson Way Suite 2","City":{"CountryRegion":"United States","Name":"Portland","Region":"WA"}}],"Gender":"Male","Concurrency":636730550970465441},{"@odata.id":"http://services.odata.org/V4/(S(zirb3lxmgzynebnvq5uhdtgx))/TripPinServiceRW/People('willieashmore')","@odata.etag":"W/\"08D61F10043BBCA1\"","@odata.editLink":"http://services.odata.org/V4/(S(zirb3lxmgzynebnvq5uhdtgx))/TripPinServiceRW/People('willieashmore')","UserName":"willieashmore","FirstName":"Willie","LastName":"Ashmore","Emails":["Willie@example.com","Willie@contoso.com"],"AddressInfo":[],"Gender":"Male","Concurrency":636730550970465441},{"@odata.id":"http://services.odata.org/V4/(S(zirb3lxmgzynebnvq5uhdtgx))/TripPinServiceRW/People('vincentcalabrese')","@odata.etag":"W/\"08D61F10043BBCA1\"","@odata.editLink":"http://services.odata.org/V4/(S(zirb3lxmgzynebnvq5uhdtgx))/TripPinServiceRW/People('vincentcalabrese')","UserName":"vincentcalabrese","FirstName":"Vincent","LastName":"Calabrese","Emails":["Vincent@example.com","Vincent@contoso.com"],"AddressInfo":[{"Address":"55 Grizzly Peak Rd.","City":{"CountryRegion":"United States","Name":"Butte","Region":"MT"}}],"Gender":"Male","Concurrency":636730550970465441},{"@odata.id":"http://services.odata.org/V4/(S(zirb3lxmgzynebnvq5uhdtgx))/TripPinServiceRW/People('clydeguess')","@odata.etag":"W/\"08D61F10043BBCA1\"","@odata.editLink":"http://services.odata.org/V4/(S(zirb3lxmgzynebnvq5uhdtgx))/TripPinServiceRW/People('clydeguess')","UserName":"clydeguess","FirstName":"Clyde","LastName":"Guess","Emails":["Clyde@example.com"],"AddressInfo":[],"Gender":"Male","Concurrency":636730550970465441},{"@odata.id":"http://services.odata.org/V4/(S(zirb3lxmgzynebnvq5uhdtgx))/TripPinServiceRW/People('keithpinckney')","@odata.etag":"W/\"08D61F10043BBCA1\"","@odata.editLink":"http://services.odata.org/V4/(S(zirb3lxmgzynebnvq5uhdtgx))/TripPinServiceRW/People('keithpinckney')","UserName":"keithpinckney","FirstName":"Keith","LastName":"Pinckney","Emails":["Keith@example.com","Keith@contoso.com"],"AddressInfo":[],"Gender":"Male","Concurrency":636730550970465441}]}



OData by Example (and imagination)

Maybe we want just the detail for the traveler: Russell Whyte

https://services.odata.org/v4/(S(0ujx3qru3djokaxrka1jknic))/TripPinServiceRW/People('russellwhyte')

{"@odata.context":"http://services.odata.org/V4/(S(0ujx3qru3djokaxrka1jknic))/TripPinServiceRW/$metadata#People/$entity","@odata.id":"http://services.odata.org/V4/(S(0ujx3qru3djokaxrka1jknic))/TripPinServiceRW/People('russellwhyte')","@odata.etag":"W/\"08D61F125DB71E23\"","@odata.editLink":"http://services.odata.org/V4/(S(0ujx3qru3djokaxrka1jknic))/TripPinServiceRW/People('russellwhyte')","UserName":"russellwhyte","FirstName":"Russell","LastName":"Whyte","Emails":["Russell@example.com","Russell@contoso.com"],"AddressInfo":[{"Address":"187 Suffolk Ln.","City":{"CountryRegion":"United States","Name":"Boise","Region":"ID"}}],"Gender":"Male","Concurrency":6367305610616581 47}

(from www.odata.org)

OData by Example (and imagination)

Or **maybe** just Russell Whyte's eMail addresses

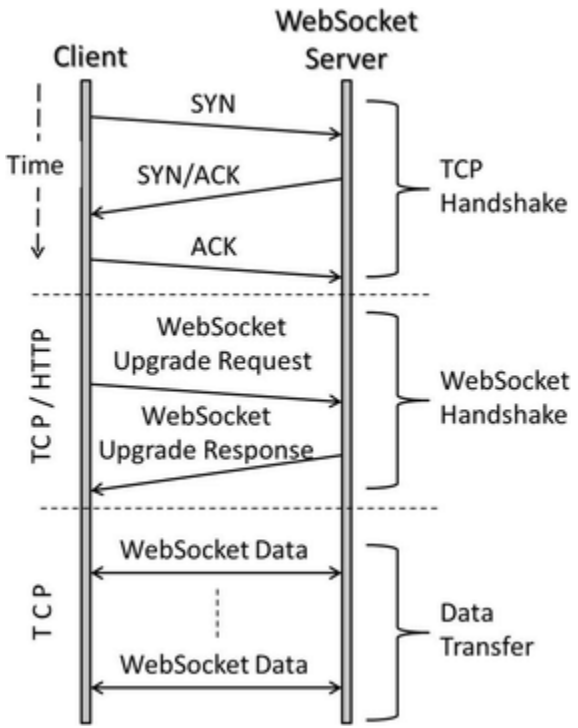
https://services.odata.org/v4/(S(0ujx3qru3djokaxrka1jknic))/TripPinServiceRW/People('russellwhyte')**?$select=Emails**

{"@odata.context":"http://services.odata.org/V4/(S(0ujx3qru3djokaxrka1jknic))/TripPinServiceRW/$metadata#People(Emails)/$entity","@odata.id":"http://services.odata.org/V4/(S(0ujx3qru3djokaxrka1jknic))/TripPinServiceRW/People('russellwhyte')","@odata.etag":"W/\"08D61F125DB71E23\"","@odata.editLink":"http://services.odata.org/V4/(S(0ujx3qru3djokaxrka1jknic))/TripPinServiceRW/People('russellwhyte')","Emails":["Russell@example.com","Russell@contoso.com"]}

While this example shows a database wrapper, pretty much any service can be imagined using SQL type operations. (from www.odata.org)

WebSockets

- Connection established and maintained via TCP
- Used for real-time updates, e.g. live chat, collaboration tools such as Google Docs
- Similarly to http:/ vs https:/, ws:/ vs wss:/ protocol, wherein the latter is TLS/SSL encrypted



Source: https://www.researchgate.net/figure/Flow-of-WebSocket-protocol-Firstly-TCP-connection-is-established-through-the-three-way_fig7_323456843

GraphQL

- Like OData, GraphQL can fetch specific attributes from a single endpoint
- Increasingly popular API query language, but REST is still king
- Stateless
- Operations include
  - Queries: for fetching data
  - Mutations: for modifying data (create, update, delete)
  - Subscriptions: GraphQL's event-driven service which provides real-time updates to the client (often on a WebSocket-driven transport layer)

GraphQL by example

```
type Query {
  hero: Character
}
```

Operation	Response
`{ hero { name friends { name } } }`	`{ "data": { "hero": { "name": "R2-D2", "friends": [ { "name": "Luke Skywalker" }, { "name": "Han Solo" }, { "name": "Leia Organa" } ] } } }`

Source: https://graphql.org/learn/queries/

SOAP (Simple Object Access Protocol)

SOAP Requests…

- Are explicitly XML based interface
- Are comprised of:
  - An “envelope” defining message structure
  - Application defined datatype encoding rules
  - “Request” and “Response” interface definitions
- Are stateless (not order dependent), but can be stateful

SOAP Request Interface via Body

Interface defined by SOAP standards:

- Requests are structured XML
- Signature described by *Web Service Definition Language* (WSDL)
- Request "block" (**body**) encapsulated within standard defined SOAP "wrapper" (**envelope**)
- Responses are structured XML

SOAP Requests:

XML **request block** encapsulated in protocol defined SOAP **envelope**

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
        xmlns:ext="urn:com.workday.report/Extract___Job_and_Person">
  <soapenv:Header/>
  <soapenv:Body>
    <ext:Execute_Report>
      <!--Optional:-->
      <ext:Report_Parameters>
        <!--Zero or more repetitions:-->
        <!--Optional:-->
        <ext:Effective_as_of_Date>?</ext:Effective_as_of_Date>
        <!--Optional:-->
        <ext:Netid>?</ext:Netid>
        <!--Optional:-->
        <ext:Active_Status>?</ext:Active_Status>
        <!--Optional:-->
      </ext:Report_Parameters>
      <!--Optional:-->
      <ext:Authentication>
        <!--Optional:-->
        <ext:Proxy_User_Name>?</ext:Proxy_User_Name>
      </ext:Authentication>
    </ext:Execute_Report>
  </soapenv:Body>
</soapenv:Envelope>
```



Salesforce SOAP Request

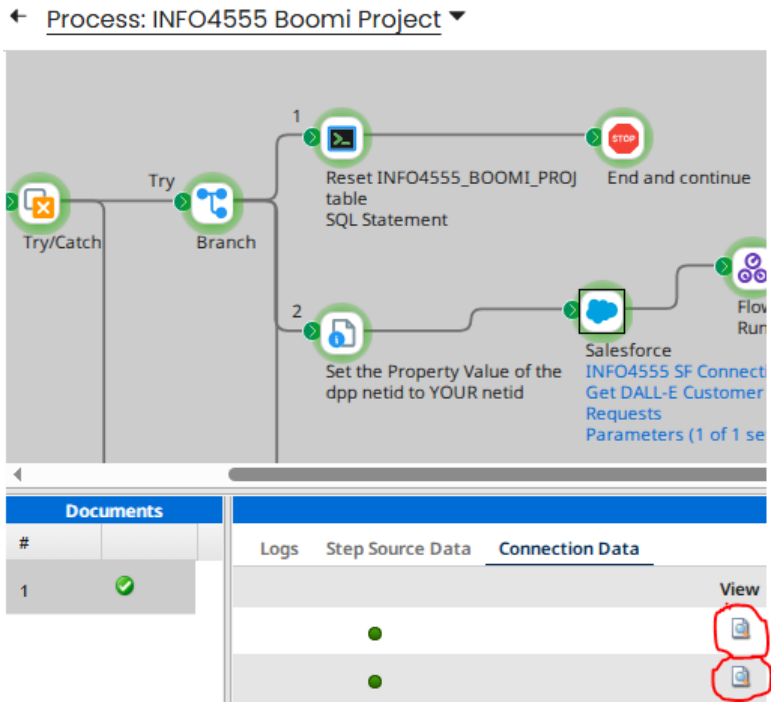
You may have noticed the URL for the Salesforce connection in the homework assignment indicates the Salesforce web service utilizes SOAP

https://test.salesforce.com/services/Soap/u/43.0

- A good example of how Boomi's built-in connectors simplify connecting to and utilizing common web services
- Can query/manipulate Salesforce data without knowing the underlying formatting or using its WSDL
- Data encapsulation with envelope occurs on the backend

Salesforce Response in Boomi

- Response is in XML format, but cuts out the SOAP envelope since it is not relevant to the data
- In Boomi, each result is returned as a separate document




```
<DALL_E_Art_Studio_Order__c>
  <Id>a4qVB000000F1pZYAS</Id>
  <OwnerId>0051Q00000TtUyTQAV</OwnerId>
  <IsDeleted>false</IsDeleted>
  <Name>PO#0162</Name>
  <CreatedDate>2025-09-08T21:13:52.000Z</CreatedDate>
  <CreatedById>0051Q00000TtUyTQAV</CreatedById>
  <LastModifiedDate>2025-09-08T21:13:52.000Z</LastModifiedDate>
  <LastModifiedById>0051Q00000TtUyTQAV</LastModifiedById>
  <SystemModstamp>2025-09-08T21:13:52.000Z</SystemModstamp>
  <First_Name__c>Fleur</First_Name__c>
  <Last_Name__c>Kirkland</Last_Name__c>
  <Request__c>A rabbit flying with butterfly wings</Request__c>
  <Assigned_To__c>pss85</Assigned_To__c>
</DALL_E_Art_Studio_Order__c>
```

The Future of APIs

- Increasing API-first approach to development
- User-authenticated agentic AI that interfaces with APIs on behalf of a person

